

Claude Code로 진짜 프로그램을 만든다

묘사하면, 내 컴퓨터에서 도는 도구가 된다.

4차에서 만든 앱, 지금 어디 있죠?

claude.ai로 만든 운세 앱 — 브라우저 안에만 있었습니다. 그리고 오늘도 R-PCCO는 그대로 씁니다.

R

역할

누가 만드나

P

목적

무엇을

C

맥락

어디서·무슨 파일

C

제약

조건·금지

O

출력

어떤 모양

오늘은 같은 R-PCCO로 내 컴퓨터에서 진짜 도는 프로그램을 만듭니다. 맥락(C)에 ‘내 컴퓨터’가 들어갈 뿐.

90분 뒤, 이렇게 달라집니다

도구 하나를 만들며 Claude Code를 통째로, 그리고 다음에도 쓸 '구조'를 손에 넣습니다.

01 · MASTERY

기능 체득

명령어·도구·권한·디버깅·git까지 한 번씩 손으로.

02 · DESIGN

레이어 설계

2단계 넘으면 쪼개 지침으로 넘겨받는 감각(2차 I/O 계약).

03 · SCAFFOLD

재사용 구조

다음 프로그램도 그대로 시작할 파일 뼈대 한 장.

브라우저를 닫으면, 아무것도 안 남았다

Artifacts는 빠르지만 모래상자입니다. 내 파일을 못 만지고, 닫으면 사라집니다.

“내 컴퓨터에서 진짜로 돌고, 내 폴더를 다루고, 남길 수 있는 것” — 그게 *Claude Code*입니다.

Artifacts vs Claude Code — 모래상자 vs 내 컴퓨터

같은 ‘AI로 만들기’지만, 도는 곳과 남는 것이 다릅니다.

구분	claude.ai Artifacts (4차)	Claude Code (6차)
어디서 도나	브라우저 모래상자	내 컴퓨터에서 진짜로
파일 개수	보통 단일 파일	여러 파일 프로젝트(레이어)
내 파일 접근	못 함	내 폴더·파일 직접
잘하는 일	빠른 웹 시제품	진짜 도구·자동화·공유까지

막히면 /, 외울 건 네 개

슬래시 한 글자로 강력해지는 단축어. 핵심 네 개에, 토큰 아끼는 /compact까지.

/help

도움말

까먹었을 때 첫 명령.

/init

CLAUDE.md 생성

새 폴더 시작할 때.

/clear

대화 비우기

새 작업 시작 시.

/permissions

권한 설정

허용·차단 규칙.

/compact

대화 압축

길어지면 요약 → 토큰 절약.

입력 트릭: **!명령** 셀 실행 · **@파일** 첨부 · **Shift+Tab** plan(계획) 모드 전환

어떤 작업이든, 이 5단계

이 흐름만 잡으면 무엇이든 시작할 수 있습니다.



핵심은 3번 — 의도 설명. 무엇을·어떻게·왜를 한 문장에 담으면 절반은 끝.

코드 말고, R-PCCO로 묘사한다

파일 변경기를 예로 — 코딩 맥락에선 ‘맥락’에 환경·파일·실행이 들어갑니다.

역할

개발자

간단한 Node 도구를 만드
는

목적

이름 변경

폴더 파일을 규칙대로 제자
리

맥락

내 컴퓨터

Windows·Node·로컬 서
버

제약

안전하게

미리보기·한글·단순하게

출력

실행 한 줄

node server.js로 열리는

Context · Tools · Permissions

이 세 단어를 이해하면 Claude Code의 모든 동작이 한 번에 보입니다.

CONTEXT

작업 메모리

지금 Claude가 아는 모든 것(대화·파일·지침).

TOOLS

손대는 능력

Read·Write·Edit·Bash — 실제로 컴퓨터에 손.

PERMISSIONS

안전벨트

위험한 작업은 '1초 읽고' 승인.

파일을 만들 때 뜨는 **권한 확인** — 빨리 가려고 다 Yes 누르지 말 것(실수 1위).

만들었으면 바로 돌려본다

Claude가 ‘로컬 서버’를 켜고(포트 3000), 나는 브라우저에서 localhost:3000으로 확인합니다.

node server.js → localhost:3000

“어, 내 컴퓨터에서 진짜 돈다!” — Artifacts엔 없던 순간. 이게 로컬 서버 테스트.

지금 애가 뭘 하는지 보인다

화면의 도구 표시를 읽으면 Claude의 행동이 다 보입니다.

Read

읽는 중

파일 내용을 확인

Write / Edit

만드는 중

파일 생성·수정

Bash

실행 중

서버·명령 돌리기

! · @

편한 기능

!!s 직접 / @파일 설명

위험한 작업은 미리보기·계획부터

파일 이름을 진짜로 바꾸는 프로그램 — 잘못 돌리면 큰일입니다.

큰 변경은 **plan 모드**: 아무것도 안 바꾸고 계획만 먼저. 그리고 항상 미리보기 → 확인 → 적용 → 되돌리기.

⚠️ 안전이 기능보다 먼저 — 되돌릴 수 없는 일은 보여주고 나서.

한 방이 아니라, 대화로 키운다

Claude Code의 진짜 힘은 기존 코드를 고쳐 나가는 반복 개선입니다.



작게 요청 → 돌려보고 → 다음. 한 번에 다 시키면 컨텍스트가 꽉 차 품질이 떨어집니다.

에러는 적이 아니라 정보다

잘 전달하면 5분 안에 해결됩니다. 디버깅 만능 5요소.

- | | | | | |
|---|--|--------------------------------------|--|--|
| <div>1
무슨 명령
뭘 했는데</div> | <div>2
에러 전체
일부 X, 전부</div> | <div>3
환경
OS·버전</div> | <div>4
언제부터
뭘 한 뒤</div> | <div>5
기대 결과
원하는 모양</div> |
|---|--|--------------------------------------|--|--|

“안 돼요, 고쳐주세요”는 정보가 0. 5요소를 채워 던지면 *Claude*가 추측 안 합니다.

한 파일에 다 엉켰다

규칙·미리보기·적용·되돌리기가
한 곳에 뒤섞였다. 2차에서 배운 거 기억나죠?

체인의 성패는 ‘접점’에 달렸다

2차 I-MRKO의 그 원칙을, 이번엔 코드로 구현합니다.

“성패는 지침이 아니라 지침 사이의 접점에 달렸다. 산문은 체인의 적.”

원칙 ①

한 단계 = 한 책임

수집은 변환 안 하고, 변환은 저장 안 한다.

원칙 ②

접점은 JSON으로

단계 사이는 구조화된 데이터로만 넘긴다.

수집 → 변환계획 → 출력

파일 변경기를 세 단계로 쪼갭니다. 사이언 JSON 계약이 흐릅니다.



한 덩어리였던 코드가 역할별 파일로 — 레이어가 눈에 보입니다. Artifacts가 못 하던 것.

접점은 포·필·범·메로 고정

단계 사이로 넘기는 데이터의 약속. 두 지침에 같은 포맷을 적어야 안 깨집니다.

포 · **FORMAT**

어떤 구조

JSON / 표 / 헤더 섹션

필 · **REQUIRED**

필수 항목

반드시 들어갈 필드

범 · **SCOPE**

범위

개수·분량·깊이 한계

메 · **METADATA**

메타정보

충돌 표시·개수 등

지침을 먼저 쓰고, 그 계약대로 만든다

스펙 우선(spec-first) — 각 단계의 입출력 계약을 MD로 먼저 못 박습니다.

specs/ 단계별 지침

- 01_수집.md — 입력 folderPath / 출력 파일목록
- 02_변환.md — 입력 파일목록+규칙 / 출력 변경계획
- 03_출력.md — 입력 변경계획 / 제자리 변경

각 지침에 적는 것

- 역할 — 이 단계가 하는 일
- 입력 / 출력 — 포·필·범·메 계약
- 책임 / 절대 안 하는 것

“이 지침대로 각 단계를 별도 파일로 구현해줘” — 묘사 한 번에 멀티파일이 만들어집니다.

지침은 **스킬로**, 중간엔 **검증을**

레이어를 더 단단하게 만드는 두 가지 — 2차 체인 원칙 그대로.

SKILL

스킬로 재사용

자주 쓰는 절차를 .claude/skills에 레시피로 저장 → 다음 프로그램에서 또 호출.

VALIDATE

검증 단계 끼우기

변환 뒤 '빈 이름·중복' 검증 한 단계 — 오류가 다음으로 번지지 않게.

어떤 프로그램이든 이 뼈대

파일 변경기뿐 아니라, 다음에 만들 무엇이든 이 구조로 시작합니다.

```
my-project/
```

```
|—— CLAUDE.md # AI가 매 세션 자동으로 읽는 지침
```

```
|—— README.md # 사람용 설명서
```

```
|—— DEVLOG.md # 개발로그 — 무엇을·언제·왜
```

```
|—— HANDOVER.md # 핸드오버 — 현재 상태·다음 할 일
```

```
|—— .env / .env.example # 비밀·설정
```

```
|—— specs/ # 단계별 지침(I/O 계약)
```

```
|—— .claude/ # settings.json · skills/
```

```
|—— src/ # 실제 코드(단계별 모듈)
```

있으면 안 무너지는 파일들

초보가 무너지는 건 실력이 아니라 구조가 없어서입니다.

파일	역할	AI 자동읽힘
CLAUDE.md	프로젝트 지침(개요·규칙·금지)	O
specs/*.md	단계별 I/O 계약	O
README / DEVLOG / HANDOVER	사람용 설명 · 기록 · 인계	—
.env / .env.example	비밀·설정 (깃 제외 / 견본)	—
.gitignore	비밀·쓰레기 깃 제외	—

언제 어떤 명령어를 쓰나

그리고 처음부터 다 만들지 말 것 — 작게 심고 키웁니다.

명령어 타이밍

- 새 폴더 → `/init`
- 새 작업 → `/clear`
- 위험 변경 전 → plan(계획) 모드 `Shift+Tab`
- 막힘 → `/help`

단계별 도입

- 최소 — CLAUDE.md·README·.gitignore
- 권장 — +DEVLOG·.env·specs
- 풀 — +HANDOVER·settings·SKILL

AI가 망쳐도 한 줄로 되돌린다

git = 시간 여행. 큰 작업 전엔 항상 커밋해 안전망을 칩니다.

습관

작업 전 **commit**

변경 전 상태를 박제. 마음에 안 들면 언제든지 복귀.

복구

git restore

AI가 100줄 고쳐놨어도 한 줄로 원상복구.

“이거 git으로 저장하고 GitHub에 올려줘” — 커밋·원격까지 Claude가 도와줍니다.

이건 URL로 남이 못 쓴다 — 왜?

내 파일을 만드는 도구의 숙명. 배포 방식은 ‘무엇을 만드느냐’가 정합니다.

내 파일을 만지나? → 예

- 배포된 웹서버는 남의 파일 못 만짐
- → **GitHub**로 코드 공유
- 남이 받아 자기 PC에서 `node server.js`

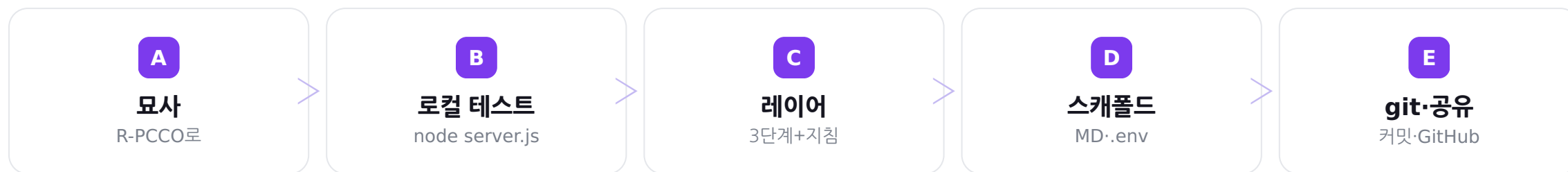
남의 파일 안 만지나? → 예

- 순수 화면·계산·조회
- → **Vercel URL 배포**(4차 운세앱)
- 남이 URL로 바로 사용

교훈: 무엇을 만드느냐가 배포 방식을 정한다.

이제 **직접** 따라 만들어봅시다

‘따라하기 명령어’를 그대로 Claude Code에 입력하며 한 사이클을 돕니다.



막히면 만능 문장: “큰 변경이니 먼저 계획만 보여줘” / “이 에러 5요소로 줄게, 고쳐줘”.

프로그램 만들기, 이 순서로

묘사 → 테스트 → 디버깅 → 레이어 → 스캐폴드 → 공유. 한 사이클.

- 1 폴더 → `claude` → `/init`
- 2 R-PCCO로 의도 설명 (작게)
- 3 로컬에서 돌려보기
- 4 2단계 이상? 레이어 + `specs/`

- 5 에러는 디버깅 5요소
- 6 `CLAUDE/README/.env`, `DEVLOG·HANDOVER`
- 7 반복작업은 SKILL
- 8 `git commit` → 공유 결정

다음 프로그램도 이 한 장으로 시작한다

오늘 만든 건 도구 하나지만, 진짜 배운 건 모든 프로그램에 깔 구조.

- ✓ 폴더 만들고 `/init` → CLAUDE.md
- ✓ R-PCCO로 의도 설명 → 최소 버전
- ✓ 로컬에서 돌려보기
- ✓ 2단계 이상? specs/ 단계별 지침
- ✓ 비밀·설정은 `.env` / `.env.example`
- ✓ DEVLOG·HANDOVER 기록
- ✓ 반복작업은 SKILL.md
- ✓ git commit → 공유 결정

“프로그램 = 코드 + 구조 파일들.” 구조부터 깔면 혼자서도 안 무너진다.

오늘부터, 이 세 가지

01

R-PCCO로 묘사

2단계 넘으면 레이어로 쪼개고 단계별 지침.

02

미리보기·plan·git

위험·되돌릴 수 없는 일은 먼저 보여주고.

03

작게 → 돌려보고 → 다음

한 방 X. 작업 바뀌면 /clear.

오늘 손으로 만진 Claude Code

하루 만에 이만큼을 직접 써봤습니다.

슬래시 명령어

Tools 관찰

권한

로컬 서버

plan 모드

디버깅

레이어 / I-O 계약

지침 **MD**

스킬

git

공유

각 레이어를 **서브에이전트**로

오늘의 ‘한 책임’ 레이어가 진짜 ‘AI 팀’이 되는 다음 단계.

15

MCP

외부 도구 연결

16

Hooks

안전장치·알림

18

서브에이전트

여러 AI 협업

가이드 고급편으로 — 같은 방식으로 더 강해집니다.

THANK
YOU

고맙습니다.

Q&A

모사 → 테스트 → 레이어 →